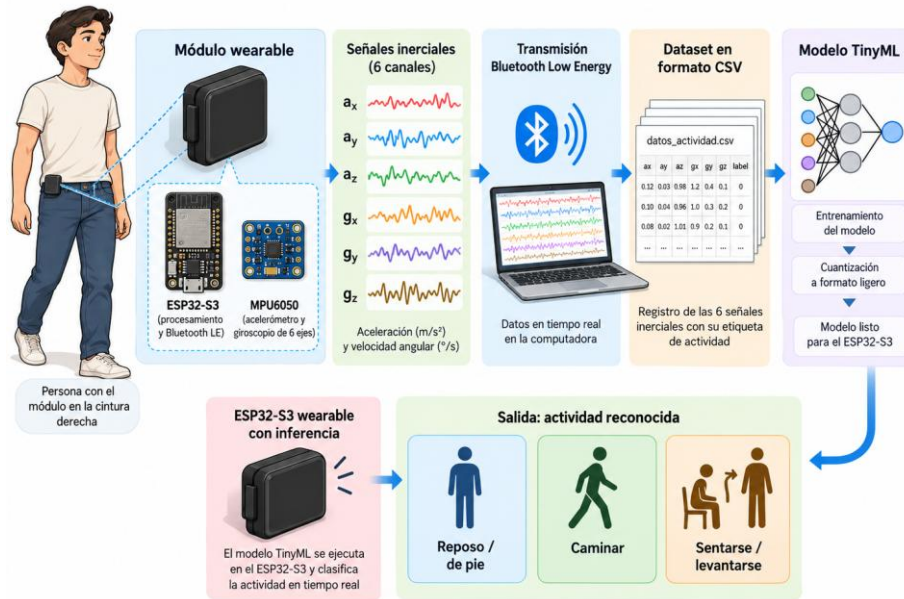
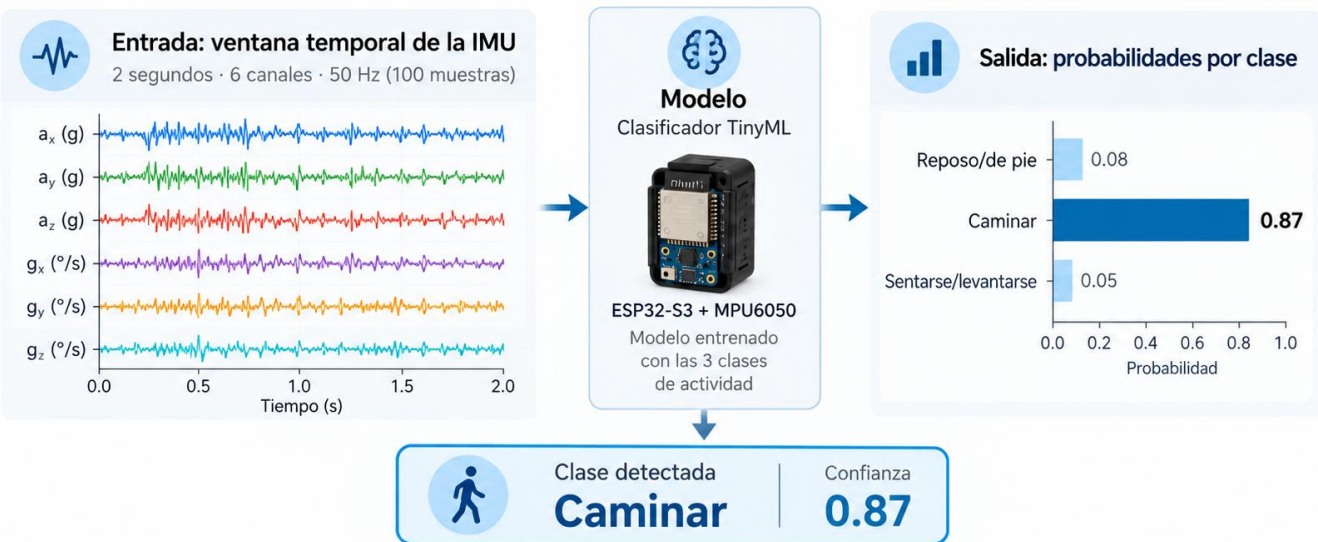


P3. Entrenamiento, validación e inferencia del modelo TinyML

Del dataset al clasificador en el ESP32-S3



Una ventana temporal entra al modelo y se convierte en una decisión de clase.



Propósito de la práctica

Transformar el dataset etiquetado en un clasificador de tres actividades, evaluar su comportamiento con datos no utilizados durante el entrenamiento y desplegar una versión optimizada para inferencia en el ESP32-S3. La laptop se utilizará como interfaz de visualización del resultado, no como sustituto de la inferencia embebida.

Resultados de aprendizaje

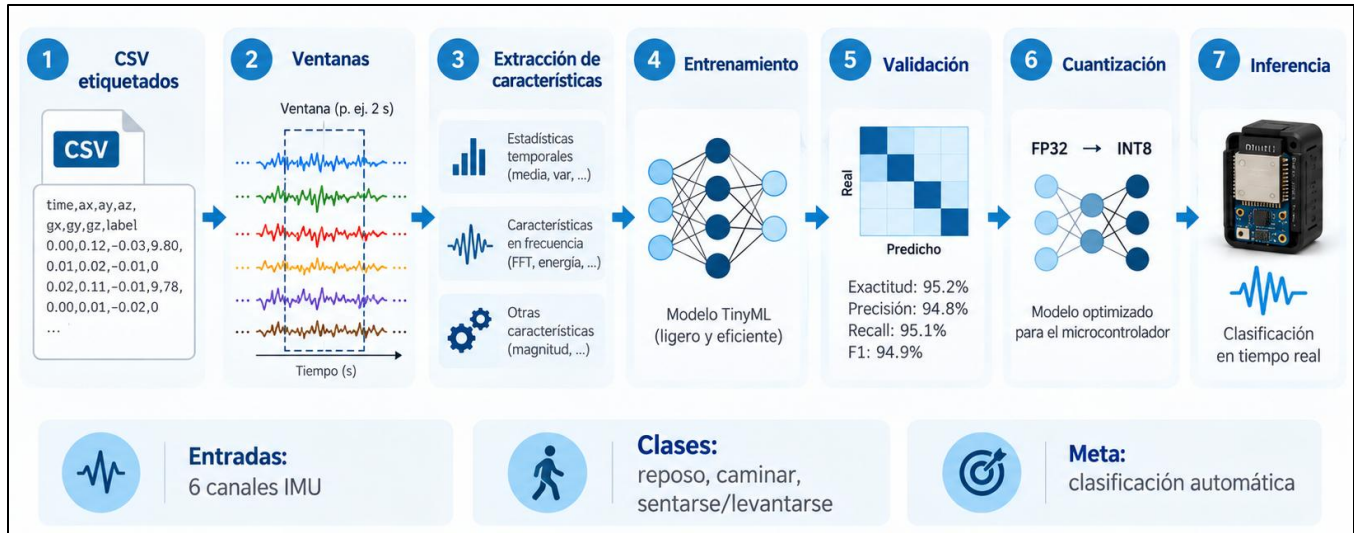
- Configurar un proyecto de clasificación de series temporales con seis canales IMU.
- Entrenar un modelo para reposo/de pie, caminar y sentarse/levantarse.
- Interpretar matriz de confusión y métricas de validación sin limitarse a una sola exactitud global.
- Comparar el comportamiento del modelo con muestras del mismo participante y de otro participante.
- Comprender la cuantización INT8 y su utilidad en microcontroladores.
- Ejecutar inferencia en el ESP32-S3 y visualizar la clase estimada y su confianza.

Materiales y preparación

- ✓ Dataset validado en P2.
- ✓ Cuenta y proyecto en Edge Impulse Studio.
- ✓ Carpeta Dataset_TinyWear y registro de capturas elaborados en P2.
- ✓ Computadora con [Arduino IDE](#) y [Python 3](#).
- ✓ Sistema TinyWear completo.

Flujo general de TinyML

El flujo parte de archivos etiquetados, genera ventanas y características, entrena un clasificador, valida su comportamiento y finalmente produce una implementación adecuada para el microcontrolador.



- 1. CSV etiquetados:** se organizan las lecturas del sensor e identifican las actividades correspondientes.
- 2. Ventanas:** se dividen las señales en segmentos cortos, por ejemplo, de 2 segundos.
- 3. Extracción de características:** se obtienen valores que describen el movimiento.
- 4. Entrenamiento:** se entrena el modelo para reconocer reposo, caminar y sentarse–levantarse.
- 5. Validación:** se evalúa el modelo con datos reservados y se analizan sus aciertos y errores.
- 6. Cuantización:** se reduce el tamaño del modelo para ejecutarlo en el microcontrolador.
- 7. Inferencia:** se identifica la actividad a partir de nuevas lecturas del sensor mediante el modelo ejecutado en el ESP32-S3.

Flujo de trabajo en Edge Impulse

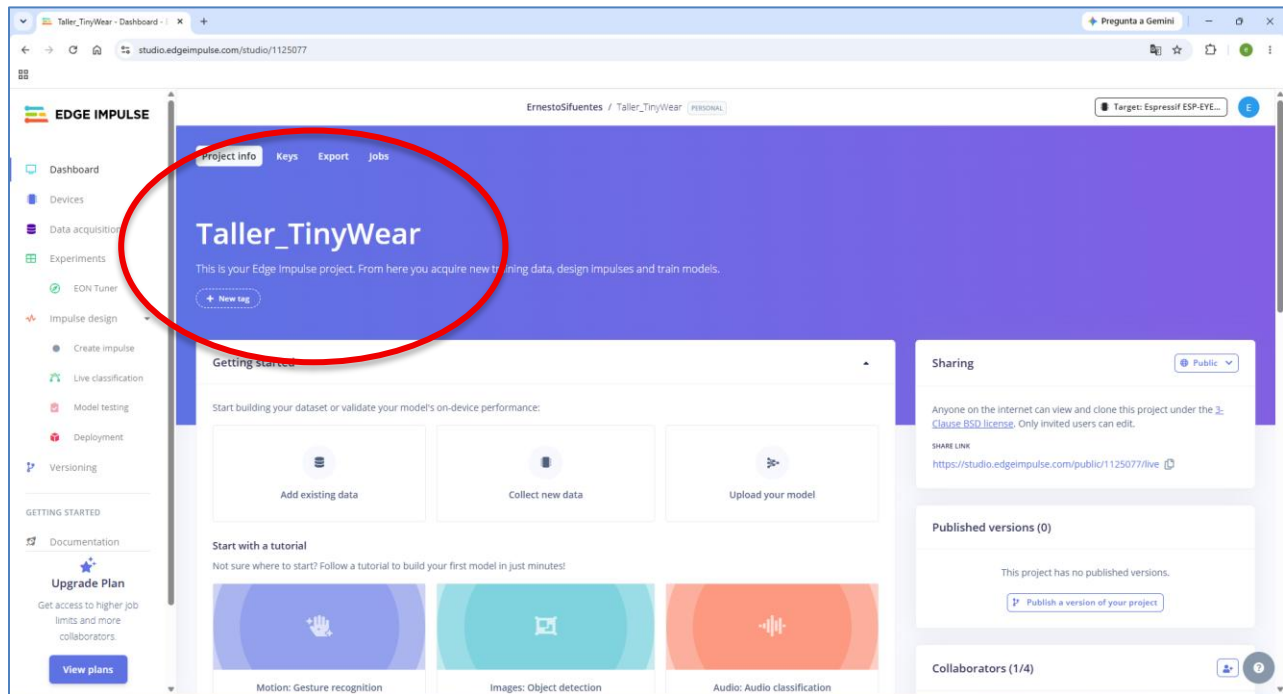
Etapas	Acción	Resultado esperado
1 . Data acquisition	Revisar dataset cargado y etiquetas.	Clases .
2 . Create impulse	Configurar ventana, señales y bloques.	Impulse definido para series temporales.
3 . Processing block	Generar características de la señal.	Feature Explorer con clases separables.
4 . Classifier	Entrenar red neuronal compacta.	Modelo con métricas iniciales.
5 . Model testing	Probar con datos no vistos.	Evidencia para decidir si pasa a P4.

Experimentos

A. Crear el proyecto y cargar el dataset

Procedimiento

- a. Acceda a la plataforma **Edge Impulse Studio** y cree un nuevo proyecto.



- b. Abra **Data acquisition** → **Dataset** → **Upload data / Upload existing data**. Edge Impulse permite cargar archivos CSV directamente desde esta sección.

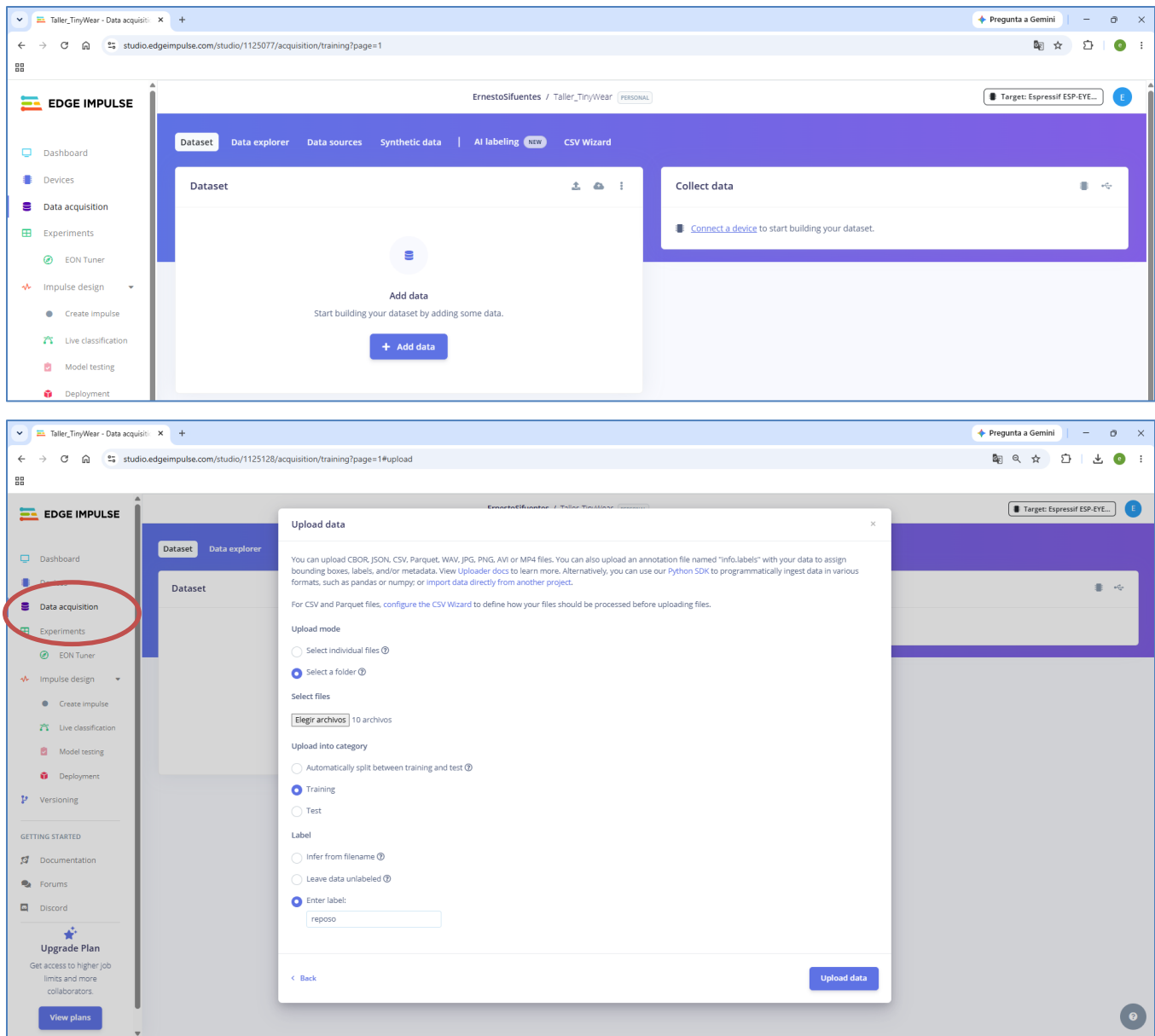
- c. Seleccione **Training** como categoría y cargue los CSV de:

```
Dataset_TinyWear/
├── entrenamiento/
│   ├── reposo/
│   ├── caminar /
│   └── sentarse_levantarse/
└── prueba/
    ├── reposo/
    ├── caminar /
    └── sentarse_levantarse/
```

- d. Seleccione **Enter Label** y escriba el nombre de la etiqueta correspondiente.

- e. La carga debe ser folder por folder:

```
entrenamiento/reposo
entrenamiento/caminar
entrenamiento/sentarse_levantarse
```

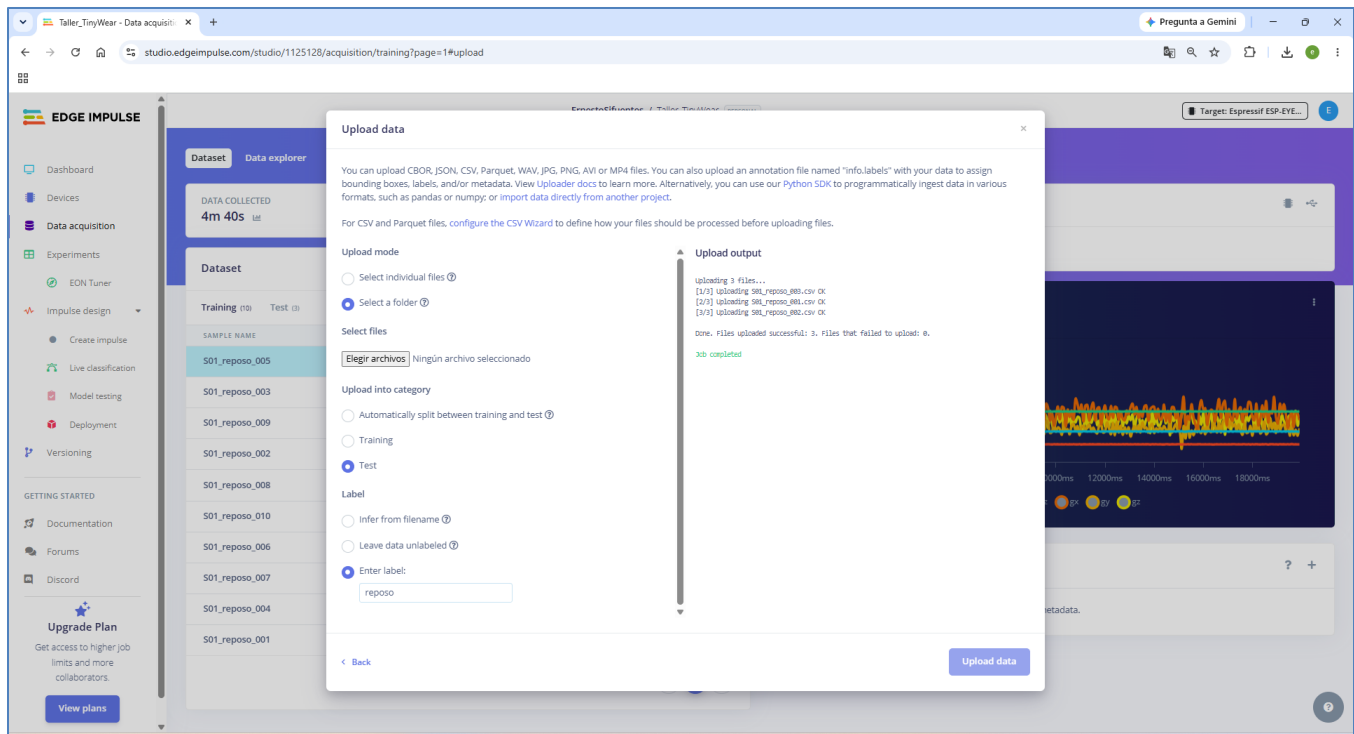


f. Confirme la configuración y seleccione **"Upload data"**.

g. Repita el procedimiento con:

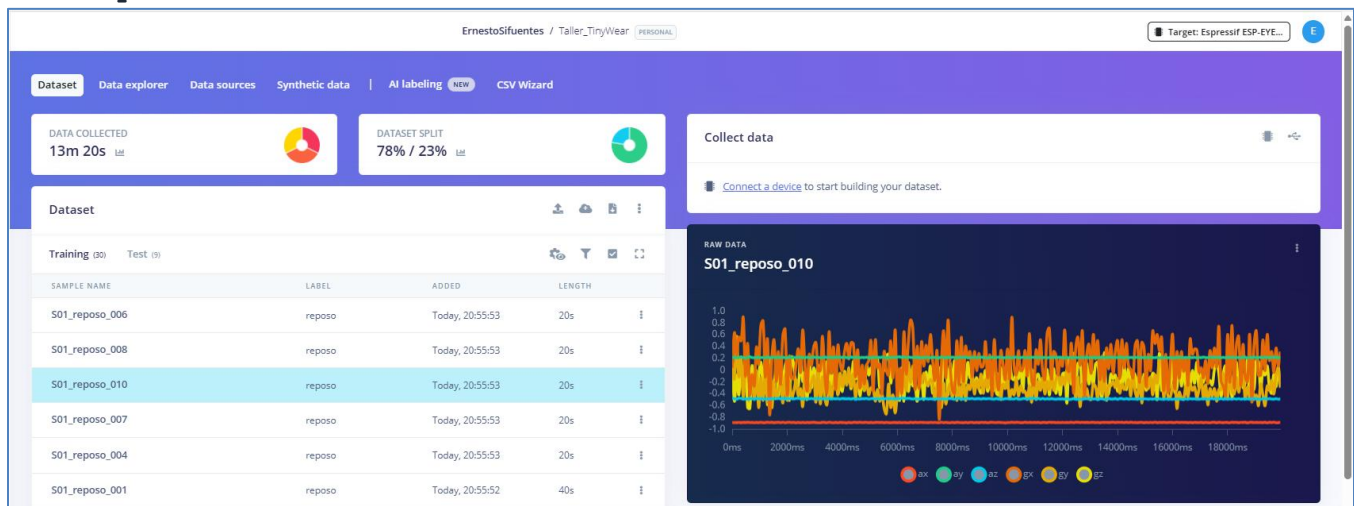
```
prueba/reposo
prueba/caminar
prueba/sentarse_levantarse
```

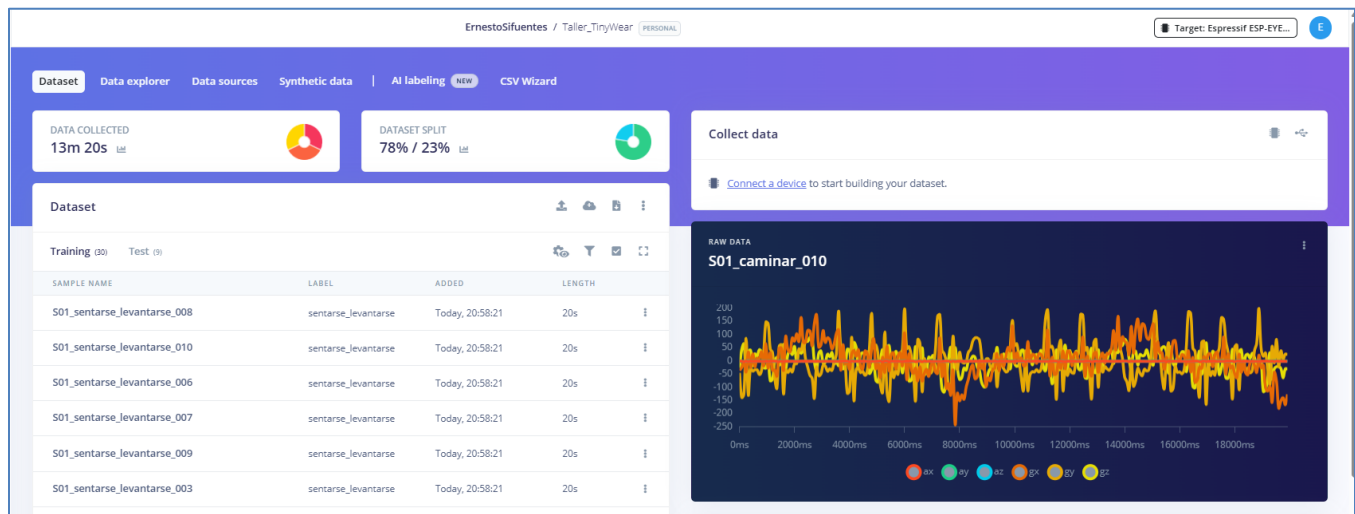
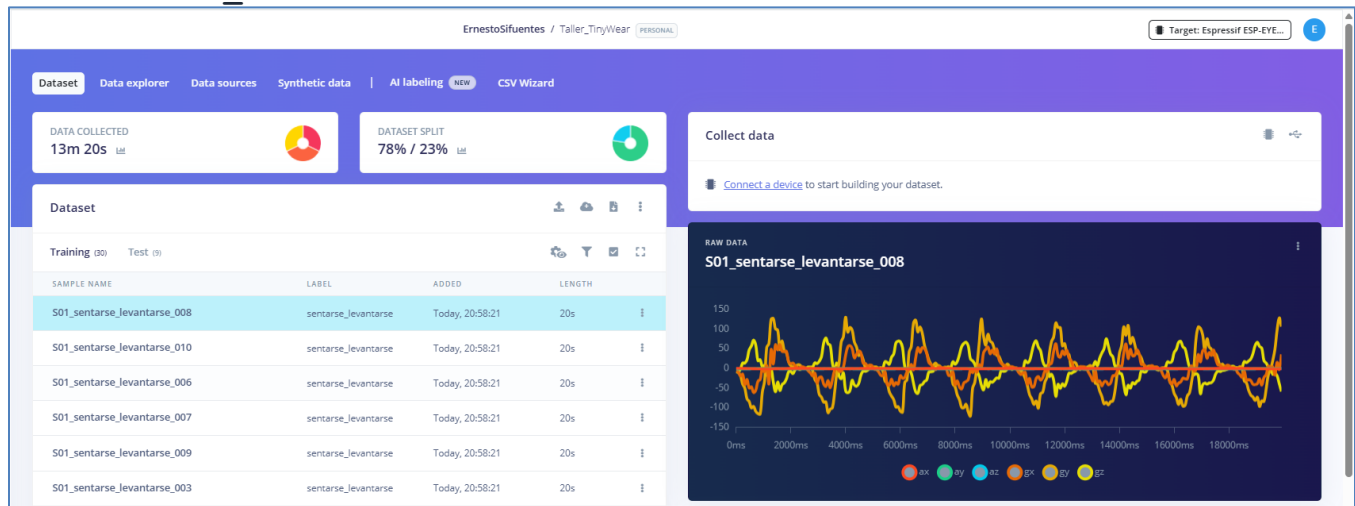
pero ahora seleccionando **Test** como categoría.



h. Finalmente, en **Data acquisition**, compruebe que aparecen las tres clases: **reposo**, **caminar** y **sentarse_levantarse**, y que los archivos de entrenamiento y de prueba estén correctamente separados.

Clase: **reposo**



Clase: **caminar**Clase: **sentarse_levantarse**

Criterios mínimos para avanzar

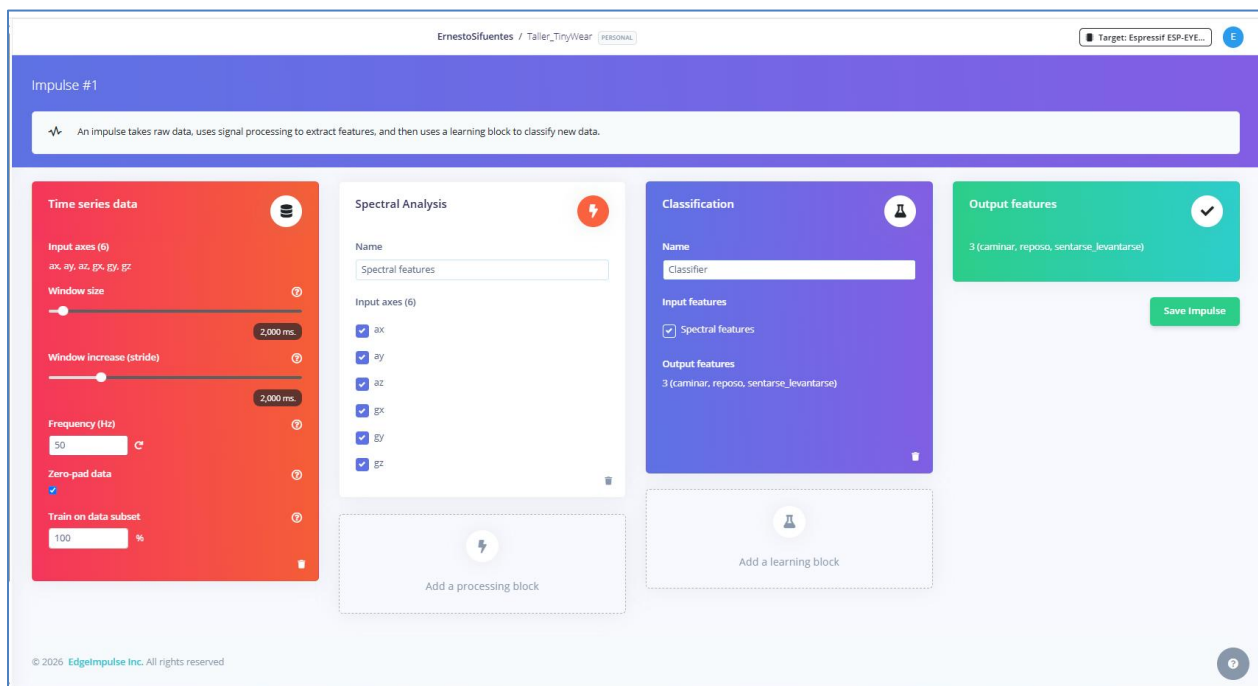
- Las tres clases están presentes.
- No hay archivos visiblemente corruptos o incompletos.
- Señales **ax**, **ay**, **az**, **gx**, **gy** y **gz**.
- Frecuencia de muestreo correcta.
- Archivos de Training y Test separados.
- Ninguna muestra con etiqueta incorrecta.

B. Configurar el impulso

El impulso define la cadena completa de procesamiento: cómo entra la señal, qué características se calculan y qué bloque de aprendizaje se entrenará.

Procedimiento

- a. En el menú lateral, seleccione **Impulse design** → **Create impulse**.
- b. En el bloque **Time series data**, verificar las señales de entrada: ax, ay, az, gx, gy y gz.
- c. Configure los parámetros temporales:
 - **Window size:** 2000 ms
 - **Window increase:** 2000 ms
 - **Frequency:** 50 Hz
 - **Zero-pad data:** activado
 - **Train on data subset:** 100 %
- d. Agregue un bloque de procesamiento para señales/vibración, preferentemente **Spectral features**.
- e. Agregue un bloque de aprendizaje de tipo **Classification**.
- f. Verifique que el clasificador reciba como entrada: **Spectral features**.
- g. Confirme las tres clases de salida: Reposo, Caminar y Sentarse_levantarse
- h. Revise que el flujo quede: **Time series data** → **Spectral Analysis** → **Classification** → **3 clases**
- i. Presione **Save Impulse**.
- j. Tome una captura de pantalla de los resultados principales.



Se propone **Spectral analysis** porque extrae características estadísticas y de frecuencia que pueden ayudar a distinguir la intensidad y repetición del movimiento.

Dentro de ese bloque, se puede comenzar con:

- **Analysis type:** FFT.
- **FFT length:** 64.
- **Filter:** None.
- **Scale axes:** 1.
- **Input decimation ratio:** 1.
- **Normalize features:** activado, para manejar las diferentes escalas del acelerómetro y el giroscopio.

Cada ventana tendrá **100 lecturas por eje y 600 valores crudos**. Esta es la configuración inicial; después se podrán comparar otras ventanas según los errores observados.

C. Extracción de características

En esta etapa, Edge Impulse transforma cada ventana temporal en un conjunto de características.

Procedimiento

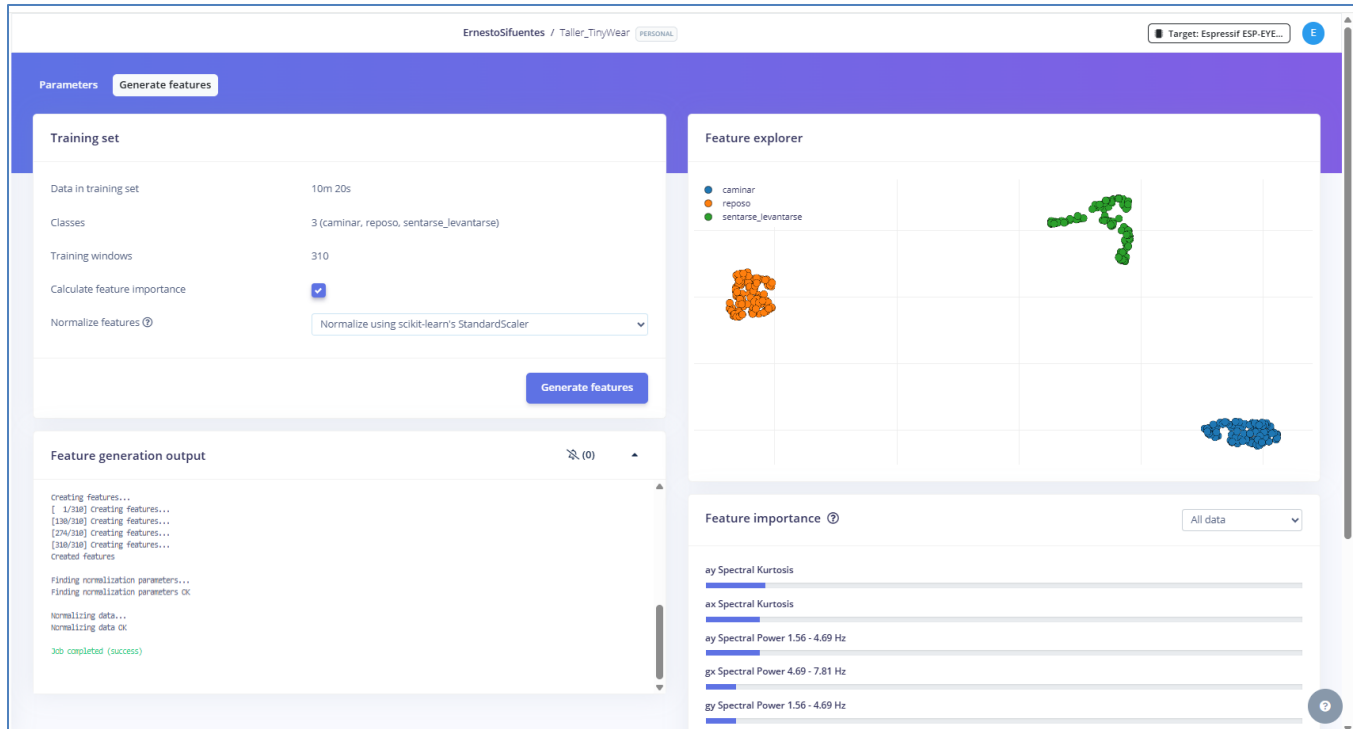
- Entre a **Impulse design** → **Spectral features**.
- Para esta primera corrida, **utilice los parámetros espectrales por defecto**.
- Presione **Save parameters**.
- Después seleccione **Generate features**.

Edge Impulse procesará todas las ventanas de entrenamiento:

$$[a_x, a_y, a_z, g_x, g_y, g_z] \rightarrow \text{características espectrales}$$

y al terminar aparecerá el **Feature Explorer**.

- Tome una captura de pantalla de los resultados principales.



Interpretación rápida

Observación	Posible interpretación	Acción sugerida
Tres clases separadas	Se observan patrones distintos entre las actividades.	Continuar con el entrenamiento y evaluar con datos reservados.
Caminar y sentarse-levantarse mezclados	Pueden existir movimientos similares o ciclos incompletos dentro de las ventanas.	Revisar las capturas y la duración de las ventanas.
Reposo mezclado con otras clases	Puede haber movimiento durante el reposo o pausas prolongadas durante las otras actividades.	Revisar las señales, las etiquetas y la ejecución de las actividades.
Puntos dispersos dentro de una clase	Puede existir variación en el ritmo del movimiento, la orientación o la sujeción del dispositivo.	Revisar la consistencia del montaje y comparar las capturas.
Puntos aislados	Puede haber una transición, una captura atípica o un error de adquisición.	Inspeccionar la señal original y repetir la captura si se identifica un error.

D. Entrenar el clasificador

Con las características generadas, se entrena un clasificador supervisado. El objetivo es obtener un modelo lo suficientemente confiable como para distinguir las tres clases.

Procedimiento

- a. Abra el bloque **Classifier**.
- b. Mantenga una configuración inicial sencilla en la primera iteración.
 - **Training cycles:** 30
 - **Learning rate:** 0.0005
 - **Learned optimizer:** desactivado
 - **Entrada:** 78 características espectrales
 - **Red:** 20 neuronas → 10 neuronas
 - **Salida:** 3 clases
- c. Presione **Start training**.
- d. Desplácese hacia abajo y observe la **matriz de confusión**.
- e. Verifique que el modelo presente una **buena precisión general** y también **dentro de cada clase**.
- f. Al terminar, revise:
 - Accuracy
 - Loss
 - Matriz de confusión
- g. Identifique la clase con mayor número de errores.
- h. Tome una captura de los resultados principales.
- i. Si los resultados no son satisfactorios, **ajuste los hiperparámetros y entrene de nuevo**.
Por ejemplo:
 - Aumente los **ciclos de entrenamiento** a **100 épocas** (en lugar de 50).
 - Agregue una **capa adicional de red neuronal**.
 - Incremente el número de **nodos por capa** a **20**.

ErnestoSifuentes / Taller_TinyWear PERSONAL Target: ESPRESSIF ESP-EYE...

Neural Network settings

Training settings

Number of training cycles 30

Use learned optimizer ☐

Learning rate 0.0005

Training processor CPU

Advanced training settings

Neural network architecture

Neural network Complete architecture / classic ML

Input layer (78 features)

Dense layer (20 neurons)

Dense layer (10 neurons)

Add an extra layer

Output layer (3 classes)

Training output

creating embeddings OK (took 2 seconds)
 calculating performance metrics...
 calculating inference time...
 calculating float32 accuracy...
 calculating int8 accuracy...
 extracting TensorBoard logs...
 compressing TensorBoard log directory...
 extracting TensorBoard logs OK

Model training complete

Job completed (success)

Model

Model version: Quantized (int8)

Last training performance (validation set)

ACCURACY 100.0% LOSS 0.02

Confusion matrix (validation set)

	CAMINAR	REPOSO	SENTARSE_LEVANTARSE
CAMINAR	100%	0%	0%
REPOSO	0%	100%	0%
SENTARSE_LEVANTARSE	0%	0%	100%
F1 SCORE	1.00	1.00	1.00

Metrics (validation set)

Metrics (validation set)

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	1.00
Weighted average Recall	1.00
Weighted average F1 score	1.00

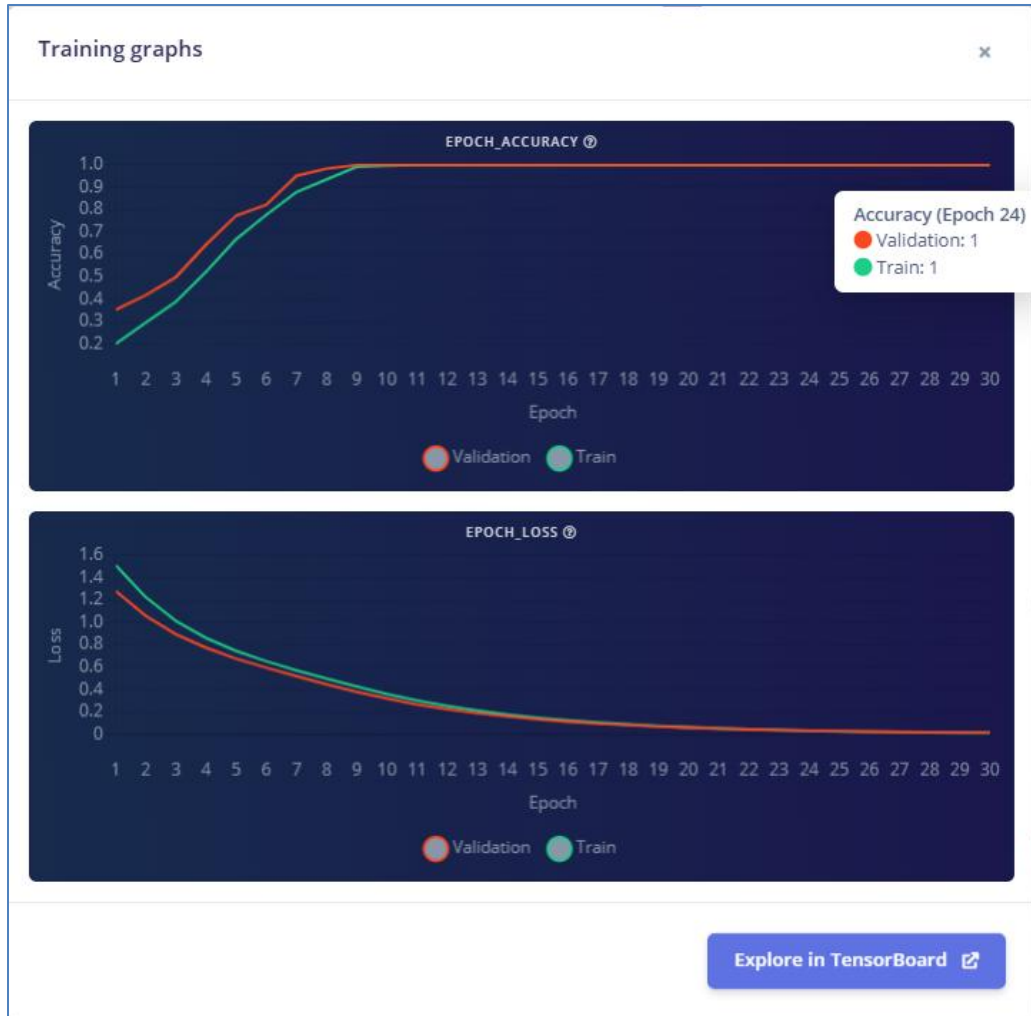
Data explorer (classified samples)

- caminar - correct
- reposito - correct
- sentarse_levantarse - correct

On-device performance

Engine: EON™ Compiler

Metric	Value
INFERENCING TIME	1 ms.
PEAK RAM USAGE	1.4K
FLASH USAGE	15.8K



Matriz de confusión

La matriz de confusión permite identificar qué actividades se confunden entre sí. En este taller es especialmente importante observar la confusión entre reposo/de pie y las transiciones de sentarse/levantarse, así como la consistencia de caminar.

Métrica	Resultado del equipo	Interpretación
Exactitud global		
Reposo/de pie		
Caminar		
Sentarse/levantarse		
Observación principal		

E. Interpretación del modelo

El modelo cuantizado **INT8** presenta una **exactitud de 100 %** y una **pérdida de 0.02 en el conjunto de validación interna**. Las métricas muestran un **AUC de 1.00** y valores ponderados de **precisión, recall y F1 de 1.00**, lo que indica una clasificación correcta de las ventanas evaluadas.

La **matriz de confusión** concentra el **100 % de las predicciones en la diagonal principal**, sin errores entre las clases **caminar, reposo y sentarse–levantarse** en este conjunto.

Las **curvas de entrenamiento** muestran que la exactitud de entrenamiento y validación alcanza el 100 % aproximadamente en la época 9 y se mantiene hasta finalizar las 30 épocas. La pérdida disminuye progresivamente en ambos conjuntos, sin una divergencia marcada entre las curvas, lo que sugiere una convergencia estable.

En el **Data Explorer** se observan tres regiones bien diferenciadas. Las ventanas mostradas se agrupan por actividad y aparecen correctamente clasificadas, en concordancia con la matriz de confusión.

En conjunto, los resultados son favorables para continuar con la **evaluación mediante datos de prueba reservados**. Esta etapa permitirá comprobar si el desempeño se mantiene en nuevas capturas y participantes que no intervinieron en el entrenamiento.

Además, Studio reporta bajos requerimientos computacionales para el modelo:

- **Tiempo estimado de inferencia:** ≈ 1 ms.
- **RAM pico estimada:** ≈ 1.4 kB.
- **Flash estimada:** ≈ 15.8 kB.

Conclusión: el modelo combina una clasificación correcta en la validación interna con una implementación compacta. Su desempeño final para TinyWear deberá confirmarse con datos independientes y pruebas de ejecución local en el ESP32-S3.

F. Validación del modelo con datos no vistos

La prueba independiente evalúa si el modelo puede clasificar ventanas que no participaron en el entrenamiento. Esta etapa es clave antes de exportar el modelo para el ESP32-S3.

Procedimiento

- a. En el menú lateral, diríjase a **Model Testing**.
- b. Verifique que las muestras que en P2 separamos como **Testing** estén cargadas.
- c. Confirme que aparezcan las tres clases:
 - o Reposo / de pie
 - o Caminar
 - o Sentarse / levantarse
- d. Presione **Classify all** o **Run model testing**.
- e. Revise la matriz de confusión del conjunto de prueba.
- f. Compare los resultados con los del entrenamiento/validación.
- g. Decida si el modelo pasa o si requiere una iteración adicional.

ErnestoSifuentes / Taller_TinyWear PERSONAL Target: Espressif ESP-EYE...

This lists all test data. You can manage this data through [Data acquisition](#).

Test data

Classify all

Set the 'expected outcome' for each sample to the desired outcome to automatically score the impulse.

SAMPLE NAME	EXPECTED OUTCOME	LENGTH	ACCURACY	RESULT
S01_sentase_levan...	sentarse_levantarse	20s	100%	10 sentarse_levantarse
S01_sentase_levan...	sentarse_levantarse	20s	100%	10 sentarse_levantarse
S01_sentase_levan...	sentarse_levantarse	20s	100%	10 sentarse_levantarse
S01_caminar_003	caminar	20s	100%	10 caminar
S01_caminar_001	caminar	20s	100%	10 caminar
S01_caminar_002	caminar	20s	100%	10 caminar
S01_reposo_002	reposo	20s	100%	10 reposo
S01_reposo_001	reposo	20s	100%	10 reposo
S01_reposo_003	reposo	20s	100%	10 reposo

Model testing output

Results

Model version: Quantized (int8)

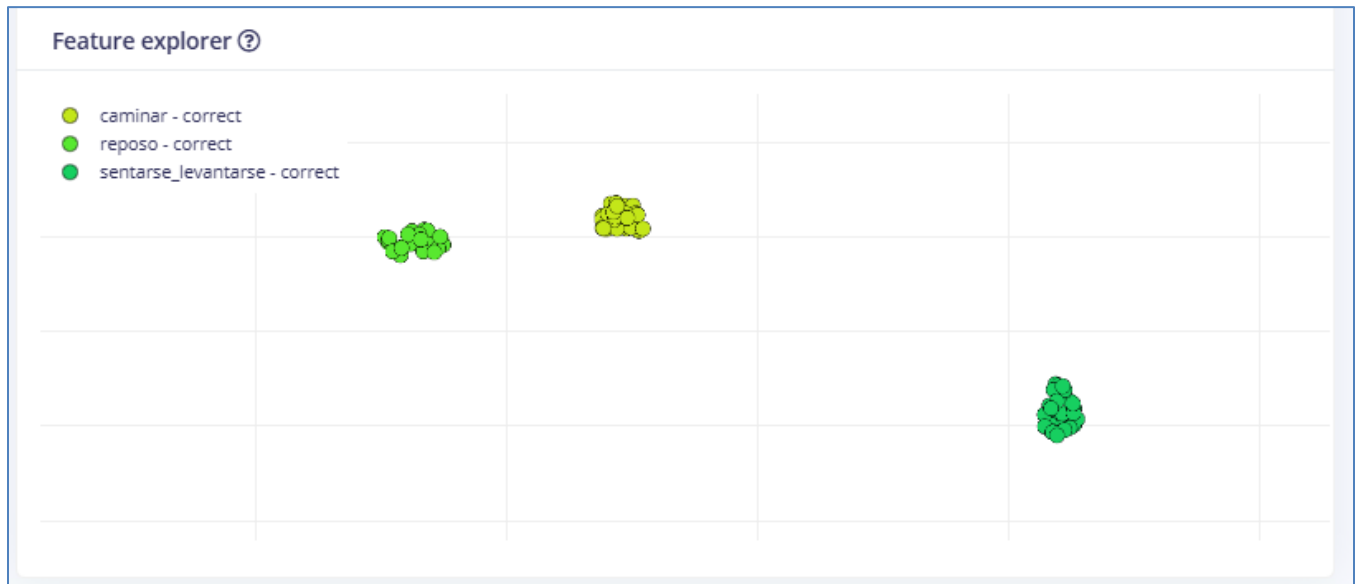
ACCURACY 100.00%

Metrics for Classifier

METRIC	VALUE
Area under ROC Curve	1.00
Weighted average Precision	1.00
Weighted average Recall	1.00
Weighted average F1 score	1.00

Confusion matrix

	CAMINAR	REPOSO	SENTARSE_LEVANTARSE	UNCERTAIN
CAMINAR	100%	0%	0%	0%
REPOSO	0%	100%	0%	0%
SENTARSE_LEVANTARSE	0%	0%	100%	0%
F1 SCORE	1.00	1.00	1.00	



G. Interpretación de ambos resultados

Tabla de equivalencias

Métrica	Entrenamiento — validación interna	Model Testing
Exactitud (<i>Accuracy</i>)	100 %	100 %
AUC	1.00	1.00
Precisión ponderada	1.00	1.00
Recall ponderado	1.00	1.00
F1 ponderado	1.00	1.00

El modelo cuantizado **INT8** alcanza una **exactitud de 100 % en Model Testing**, coincidente con la obtenida en la validación interna. Las métricas de precisión, recall y F1 ponderados presentan valores de **1.00**, lo que indica una clasificación correcta de las ventanas del conjunto de prueba evaluado.

La **matriz de confusión** muestra un **100 % de reconocimiento** para las tres actividades: **caminar, reposo y sentarse-levantarse**. No se observan confusiones entre clases ni ventanas clasificadas como inciertas.

En el **Feature Explorer** se observan **tres grupos compactos y bien separados**, con todos los puntos identificados como correctos. Esta distribución es consistente con los resultados de la matriz de confusión y sugiere que las características extraídas permiten distinguir las actividades en las muestras evaluadas.

La coincidencia entre ambos resultados indica que **no se observa una disminución del desempeño en este conjunto de prueba**. Para comprobar la generalización a otras personas y condiciones de uso, se requiere ampliar la evaluación con nuevas capturas y participantes no utilizados durante el entrenamiento.

Conclusión: el modelo presenta resultados favorables para continuar con el despliegue y la validación física en el **ESP32-S3**, donde deberá comprobarse el reconocimiento de las actividades durante su ejecución local.

H. Modelo para microcontrolador

Una vez validado el modelo, se genera una versión optimizada para el ESP32-S3. La cuantización INT8 reduce el tamaño del modelo y el costo de cálculo, lo que permite la inferencia local con recursos limitados.

Salida del clasificador

Para cada ventana, el modelo produce una probabilidad por clase. La decisión inicial puede tomarse seleccionando la clase con mayor probabilidad.

$$clase\ estimada = \arg \max_k p(k | ventana)$$

I. Crear la biblioteca para inferencia

Procedimiento

- a. En Edge Impulse, entre a **Deployment**.
- b. Seleccione: **Create library** → **Arduino library**. Esto permite exportar el modelo como biblioteca de Arduino para integrarlo en un sketch propio.
- c. Para nuestro caso:
 - **Quantized (int8)**
 - **EON Compiler** activado, si está disponible

La versión INT8 es la opción lógica para TinyML porque facilita la inferencia embebida.
- d. Presione **Build**.
- e. Edge Impulse descargará, en su computadora, un archivo ***.zip** con la biblioteca.

Configure your deployment

You can deploy your impulse to any device. This makes the model run without an internet connection, minimizes latency, and runs with minimal power consumption. [Read more.](#)

Deployment target

Arduino library
An Arduino library with examples that runs on most Arm-based Arduino development boards.

Inference engine

EON™ Compiler
Same accuracy, 54% less RAM, 56% less ROM.

Model optimizations and performance

Model optimizations can increase on-device performance but may reduce accuracy. Performance estimate for Espressif ESP-EYE (ESP32 240MHz) - [Change target](#)

Quantized (int8)

Selected

	SPECTRAL FEATURES		CLASSIFIER	TOTAL
LATENCY	47 ms.		1 ms.	48 ms.
RAM	3.2K		1.4K	3.2K
FLASH	-		15.8K	-
ACCURACY				100.00%

Unoptimized (float32)

Select

	SPECTRAL FEATURES		CLASSIFIER	TOTAL
LATENCY	47 ms.		2 ms.	49 ms.
RAM	3.2K		1.6K	3.2K
FLASH	-		17.7K	-
ACCURACY				100.00%

Build

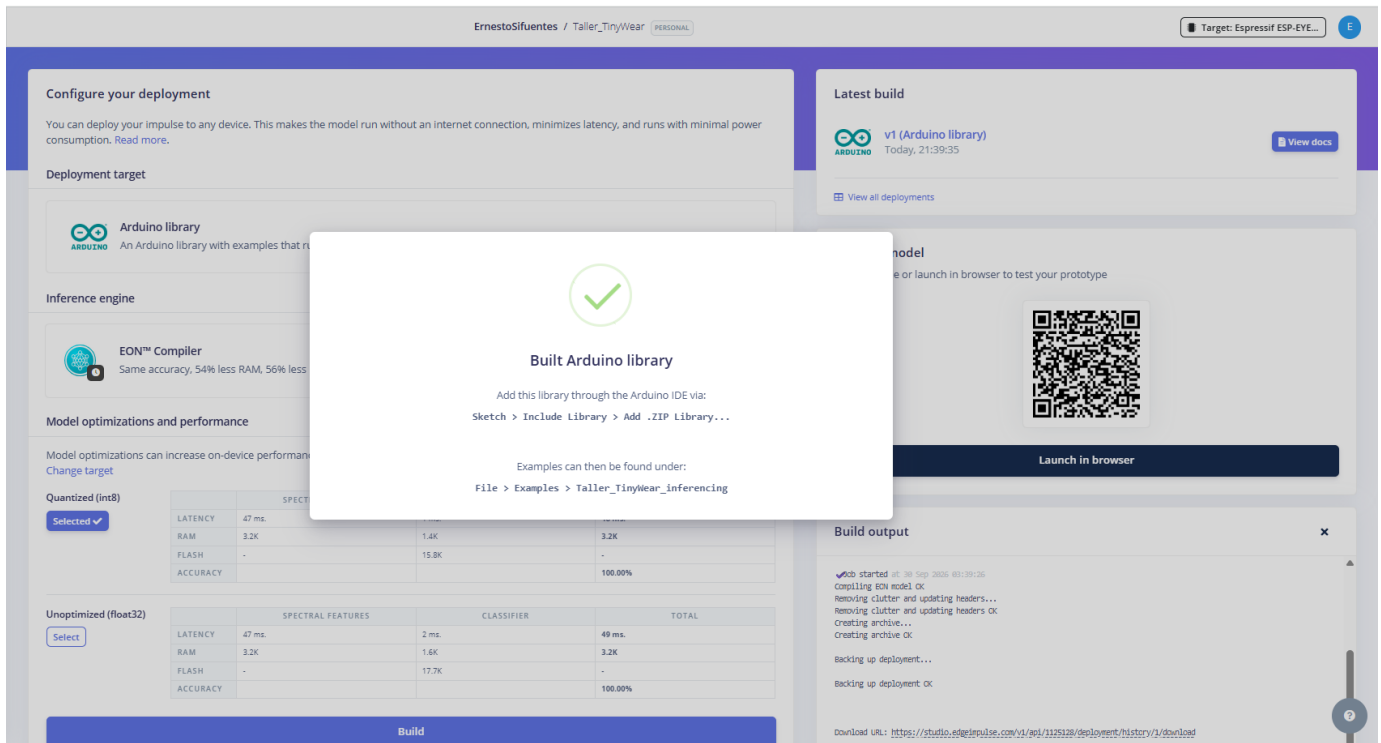
Latest build

No deployments yet

Run this model

Scan QR code or launch in browser to test your prototype

Launch in browser

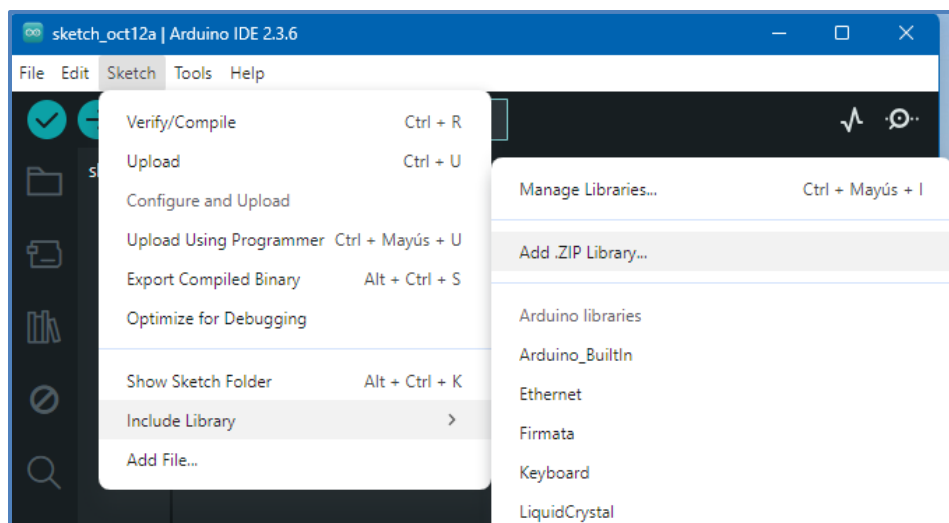


J. Instalar la biblioteca en Arduino IDE

Procedimiento

- Abra Arduino IDE.
- Vaya a **Sketch >> Include Library >> Add .ZIP Library...**
- Seleccione el **ZIP** descargado desde Edge Impulse.

Ejemplo: ei-taller_tinywear-arduino-1.0.1-impulse-#1.zip



K. Integrar inferencia con TinyWear

En esta etapa se carga el modelo INT8 en el ESP32-S3. El dispositivo adquiere las señales del MPU6050 y reconoce **reposo/de pie, caminar y sentarse-levantarse**. La computadora recibe las predicciones por Wi-Fi directo y UDP, muestra las tres probabilidades y guarda los resultados para la validación física.

Procedimiento

- a. En **Tools** → **Board**, seleccione ESP32-S3
- b. En **Tools** → **Port**, seleccione el puerto COM correspondiente.
- c. Se mantiene la conexión utilizada en P1 y P2:

MPU6050	ESP32-S3
VCC	3.3 V
GND	GND
SDA	GPIO 8
SCL	GPIO 9

El sensor debe conservar la misma orientación utilizada durante la adquisición del dataset.

- d. Abra el programa **TinyWear_Inferencia_WIFI.ino**
Se recomienda abrir el archivo `.ino` desde su carpeta y conservar sus archivos `.cpp` y `.h`.
- e. En **TinyWearConfig.h** compruebe `SDA_PIN = 8` y `SCL_PIN = 9`. Se asigna un **KIT_ID** distinto a cada equipo. Con `KIT_ID = 1` se genera la red **TinyWear-1**; con `KIT_ID = 2`, **TinyWear-2**.
- f. La cabecera utilizada en **TinyWearApp.cpp** debe coincidir exactamente con la biblioteca entregada: `#include <Taller_TinyWear_inferencing.h>`
- g. Compile el programa.
- h. Cargue el programa en el ESP32-S3.
- i. Abra el Monitor Serial a 115200 bit/s.

Funcionamiento esperado: se obtiene una decisión cada 2 s, a partir de 100 lecturas de seis ejes a 50 Hz. El Monitor Serial muestra la clase detectada, la confianza y los tiempos de DSP, NN y total. La inferencia continúa aunque no haya una computadora conectada.

Ejemplo de una captura

Output	Serial Monitor X
Message (Enter to send message to 'ESP32S3 Dev Module' on 'COM3')	
Ventana 48	caminar 92.6 % DSP 5.481 ms NN 0.441 ms total 6.138 ms
Ventana 49	incierto 39.5 % DSP 6.018 ms NN 0.452 ms total 6.668 ms
Ventana 50	caminar 99.6 % DSP 5.479 ms NN 0.439 ms total 6.136 ms
Ventana 51	sentarse_levantarse 82.4 % DSP 5.477 ms NN 0.454 ms total 6.123 ms
Ventana 52	sentarse_levantarse 91.8 % DSP 5.478 ms NN 0.440 ms total 6.137 ms
Ventana 53	reposo 97.7 % DSP 5.476 ms NN 0.446 ms total 6.116 ms
Ventana 54	sentarse_levantarse 83.6 % DSP 5.480 ms NN 0.438 ms total 6.137 ms
Ventana 55	sentarse_levantarse 96.9 % DSP 5.480 ms NN 0.445 ms total 6.117 ms
Ventana 56	sentarse_levantarse 95.3 % DSP 5.476 ms NN 0.441 ms total 6.139 ms
Ventana 57	sentarse_levantarse 91.0 % DSP 5.477 ms NN 0.447 ms total 6.142 ms
Ventana 58	sentarse_levantarse 73.4 % DSP 5.478 ms NN 0.438 ms total 6.136 ms
Ventana 59	sentarse_levantarse 94.9 % DSP 5.478 ms NN 0.441 ms total 6.135 ms
Ventana 60	sentarse_levantarse 94.1 % DSP 5.476 ms NN 0.438 ms total 6.135 ms
Ventana 61	sentarse_levantarse 94.5 % DSP 5.475 ms NN 0.441 ms total 6.136 ms
Ventana 62	sentarse_levantarse 90.2 % DSP 5.477 ms NN 0.441 ms total 6.144 ms

L. Prueba funcional

Se comprueba el comportamiento de TinyWear con capturas nuevas y actividades conocidas. Se mantienen el montaje de P2 y un ritmo de movimiento semejante al utilizado al construir el dataset.

Procedimiento

- Coloque TinyWear en la misma posición utilizada durante la construcción del dataset.
- Conecte la computadora a la red Wi-Fi **TinyWear-1**, contraseña: **TinyWear2026**. Si se cambia el `KIT_ID`, cambia el número del SSID. El aviso de Windows «sin Internet» es normal.
- Abra el programa **Interfaz_TinyWear_WIFI.py** (En la carpeta Python, ejecute **Iniciar_TinyWear.bat**).
- En la interfaz, pulse **Conectar**. IP predeterminada: **192.168.4.1**. Puerto: **5005**. Se espera una decisión aproximadamente cada dos segundos.
- Pulse **Iniciar CSV** para registrar, indicar sujeto, seleccionar la actividad real. Los cambios de actividad marcan inicialmente ventanas de transición que no cuentan en la exactitud observada.
- Se observan la actividad detectada, la confianza y las tres probabilidades. La clasificación se ejecuta en el ESP32-S3; Python recibe, muestra y registra las predicciones.

Ejemplo ilustrativo de una ventana de reposo

Actividad	Probabilidad mostrada
Reposo / de pie	96.8 %
Caminar	2.4 %
Sentarse–levantarse	0.8 %

En este ejemplo, la actividad detectada sería **Reposo / de pie** y la confianza **96.8 %**.

Criterio de decisión: se selecciona la clase con mayor puntuación cuando alcanza **0.60 o 60 %**. Si ninguna alcanza ese umbral, se muestra **Incierto**, que es una decisión de rechazo. La confianza corresponde a una ventana; la exactitud observada se calcula mediante la comparación con la actividad real.

- g. Permanezca en **reposo/de pie**. Una vez establecida la postura, se selecciona **Reposo / de pie** en **Actividad real**. Se espera a que las filas dejen de mostrar "**Transición**" y se mantiene la actividad durante **al menos 20 s**.
- h. Al cambiar de actividad, se selecciona "**Sin etiqueta**". Comience a caminar y, cuando el movimiento sea continuo, seleccione **Caminar**. Tras la transición inicial se registran **al menos 20 s** de marcha.
- i. Se vuelve a **Sin etiqueta** y se comienzan ciclos completos de **sentarse y levantarse**. Cuando las repeticiones son continuas, se selecciona **la transición Sentarse–levantarse y se registra al menos 20 s** después de esta.
- j. Guarde una captura de la interfaz por actividad y apunte confusiones, decisiones inciertas y condiciones del montaje. Realice **dos rondas completas**, manteniendo intervalos similares entre las tres actividades.
- k. Al terminar, pulse **Detener CSV**. Compruebe el mensaje de registro guardado y abra el archivo para verificar que contenga filas. La columna **fuentes** debe indicar **ESP32_S3**.
- l. Si se evalúa otro participante, se detiene primero el CSV anterior, se cambia **Sujeto** a **S02**, se pulsa **Reiniciar conteo** y se inicia un archivo nuevo. Para explorar la generalización entre personas, se utiliza un participante ajeno al entrenamiento.

Etiqueta real: la actividad se asigna a partir de lo que realiza el participante. Durante los cambios se mantiene **Sin etiqueta**; la interfaz excluye además las primeras ventanas mediante un margen temporal aproximado. Esa exclusión no detecta el movimiento real, por lo que se revisa la correspondencia entre la etiqueta y la actividad.

Evaluación: las ventanas con actividad real estable cuentan como aciertos o errores. Una decisión **Incierto** cuenta como error cuando existe una etiqueta real. Las filas **Sin etiqueta** y **Transición** se guardan, pero quedan fuera del cálculo de exactitud.

M. Resultados de las pruebas realizadas

Reposo de pie

TinyWear | Inferencia por Wi-Fi

TinyWear | Inferencia local

Wi-Fi conectado · UDP registrado

Kit 1 · INT8 · 50 Hz · ventana de 2 s · 78 características · umbral 0.60

Conexión Wi-Fi / UDP

IP del ESP32: 192.168.4.1

Puerto local: 5005

Conectar

Desconectar

Modelo con normalización incorporada. Red TinyWear-1 · UDP 5005.

Decisión del ESP32-S3

Reposo / de pie

98.8 %

Puntuación máxima del modelo; no representa la exactitud de la sesión.

Caminar

1.2 %

Reposo / de pie

98.8 %

Sentarse-levantarse

0.0 %

DSP: 5.411 ms Inferencia: 0.449 ms Procesamiento total: 6.068 ms

Validación física y registro CSV

Sujeto: S01

Actividad real: Sin etiqueta

Iniciar CSV

Reiniciar conteo

Las primeras ventanas después de cambiar la actividad real se marcan como transición.

Registro CSV detenido

Ventanas recibidas: 23 · Evaluadas: 0 · Exactitud observada: — · Inciertas: 0 · Saltos de secuencia: 0

Sensor: OK · Ventana: 97/100 · Errores I²C: 0 · Ventanas descartadas: 0 · Cola: 0 · Jitter máximo: 0.01 ms

Hora	Ventana	Actividad detectada	Confianza	Actividad real	Evaluación	Total ms
16:22:27	33	Caminar	99.2 %	Sin etiqueta	Sin evaluar	6.068
16:22:29	34	Caminar	98.0 %	Sin etiqueta	Sin evaluar	6.256
16:22:31	35	Caminar	83.2 %	Sin etiqueta	Sin evaluar	6.068
16:22:33	36	Reposo / de pie	98.8 %	Sin etiqueta	Sin evaluar	6.094
16:22:35	37	Reposo / de pie	99.6 %	Sin etiqueta	Sin evaluar	6.084
16:22:37	38	Reposo / de pie	98.8 %	Sin etiqueta	Sin evaluar	6.098
16:22:39	39	Reposo / de pie	98.8 %	Sin etiqueta	Sin evaluar	6.068

TinyWear · ELECTRO 2026 · Ernesto Sifuentes de la Hoya

Caminar

TinyWear | Inferencia por Wi-Fi

TinyWear | Inferencia local

Wi-Fi conectado · UDP registrado

Kit 1 · INT8 · 50 Hz · ventana de 2 s · 78 características · umbral 0.60

Conexión Wi-Fi / UDP

IP del ESP32: 192.168.4.1

Puerto local: 5005

Conectar

Desconectar

Modelo con normalización incorporada. Red TinyWear-1 · UDP 5005.

Decisión del ESP32-S3

Caminar

91.0 %

Puntuación máxima del modelo; no representa la exactitud de la sesión.

Caminar

91.0 %

Reposo / de pie

8.6 %

Sentarse-levantarse

0.0 %

DSP: 5.609 ms Inferencia: 0.459 ms Procesamiento total: 6.268 ms

Validación física y registro CSV

Sujeto: S01

Actividad real: Sin etiqueta

Iniciar CSV

Reiniciar conteo

Las primeras ventanas después de cambiar la actividad real se marcan como transición.

Registro CSV detenido

Ventanas recibidas: 48 · Evaluadas: 0 · Exactitud observada: — · Inciertas: 5 · Saltos de secuencia: 2

Sensor: OK · Ventana: 97/100 · Errores I²C: 0 · Ventanas descartadas: 0 · Cola: 0 · Jitter máximo: 0.01 ms

Hora	Ventana	Actividad detectada	Confianza	Actividad real	Evaluación	Total ms
16:23:21	60	Incierto	59.0 %	Sin etiqueta	Sin evaluar	6.566
16:23:23	61	Caminar	62.1 %	Sin etiqueta	Sin evaluar	6.297
16:23:25	62	Incierto	43.4 %	Sin etiqueta	Sin evaluar	6.064
16:23:27	63	Incierto	48.4 %	Sin etiqueta	Sin evaluar	6.067
16:23:29	64	Incierto	56.6 %	Sin etiqueta	Sin evaluar	6.474
16:23:31	65	Caminar	90.2 %	Sin etiqueta	Sin evaluar	6.296
16:23:33	66	Caminar	91.0 %	Sin etiqueta	Sin evaluar	6.268

TinyWear · ELECTRO 2026 · Ernesto Sifuentes de la Hoya

Sentarse_levantarse

TinyWear | Inferencia por Wi-Fi

TinyWear | Inferencia local

Wi-Fi conectado · UDP registrado

Kit 1 · INT8 · 50 Hz · ventana de 2 s · 78 características · umbral 0.60

Conexión Wi-Fi / UDP

IP del ESP32: 192.168.4.1

Puerto local: 5005

Conectar

Desconectar

Modelo con normalización incorporada. Red TinyWear-1 · UDP 5005.

Decisión del ESP32-S3

Sentarse-levantarse

98.8 %

Puntuación máxima del modelo; no representa la exactitud de la sesión.

Caminar

0.8 %

Reposo / de pie

0.4 %

Sentarse-levantarse

98.8 %

DSP: 5.604 ms Inferencia: 0.446 ms Procesamiento total: 6.250 ms

Validación física y registro CSV

Sujeto: S01

Actividad real: Sin etiqueta

Iniciar CSV

Reiniciar conteo

Las primeras ventanas después de cambiar la actividad real se marcan como transición.

Registro CSV detenido

Ventanas recibidas: 34 · Evaluadas: 0 · Exactitud observada: — · Inciertas: 1 · Saltos de secuencia: 1

Sensor: OK · Ventana: 97/100 · Errores I²C: 0 · Ventanas descartadas: 0 · Cola: 0 · Jitter máximo: 0.01 ms

Hora	Ventana	Actividad detectada	Confianza	Actividad real	Evaluación	Total ms
16:22:51	45	Reposo / de pie	99.6 %	Sin etiqueta	Sin evaluar	6.060
16:22:53	46	Incierto	59.0 %	Sin etiqueta	Sin evaluar	6.681
16:22:55	47	Sentarse-levantarse	97.3 %	Sin etiqueta	Sin evaluar	6.066
16:22:57	48	Sentarse-levantarse	97.3 %	Sin etiqueta	Sin evaluar	6.346
16:22:59	49	Sentarse-levantarse	94.5 %	Sin etiqueta	Sin evaluar	6.671
16:23:01	50	Sentarse-levantarse	94.9 %	Sin etiqueta	Sin evaluar	6.069
16:23:03	51	Sentarse-levantarse	98.8 %	Sin etiqueta	Sin evaluar	6.250

TinyWear · ELECTRO 2026 · Ernesto Sifuentes de la Hoya

Incierto

TinyWear | Inferencia por Wi-Fi

TinyWear | Inferencia local

Wi-Fi conectado · UDP registrado

Kit 1 · INT8 · 50 Hz · ventana de 2 s · 78 características · umbral 0.60

Conexión Wi-Fi / UDP

IP del ESP32: 192.168.4.1 Puerto local: 5005 Conectar Desconectar

Modelo con normalización incorporada. Red TinyWear-1 · UDP 5005.

Decisión del ESP32-S3

Incierto 54.3 %

Puntuación máxima del modelo; no representa la exactitud de la sesión.

Caminar 43.4 %

Reposo / de pie 2.3 %

Sentarse-levantarse 54.3 %

DSP: 5.422 ms Inferencia: 0.471 ms Procesamiento total: 6.088 ms

Validación física y registro CSV

Sujeto: S01 Actividad real: Sin etiqueta Iniciar CSV Reiniciar conteo

Las primeras ventanas después de cambiar la actividad real se marcan como transición.

Registro CSV detenido

Ventanas recibidas: 59 · Evaluadas: 0 · Exactitud observada: — · Inciertas: 7 · Saltos de secuencia: 3

Sensor: OK · Ventana: 97/100 · Errores I²C: 0 · Ventanas descartadas: 0 · Cola: 0 · Jitter máximo: 0.01 ms

Hora	Ventana	Actividad detectada	Confianza	Actividad real	Evaluación	Total ms
16:23:45	72	Reposo / de pie	98.8 %	Sin etiqueta	Sin evaluar	6.316
16:23:47	73	Reposo / de pie	99.2 %	Sin etiqueta	Sin evaluar	6.088
16:23:49	74	Caminar	97.7 %	Sin etiqueta	Sin evaluar	6.141
16:23:51	75	Sentarse-levantarse	71.5 %	Sin etiqueta	Sin evaluar	6.519
16:23:53	76	Sentarse-levantarse	82.4 %	Sin etiqueta	Sin evaluar	6.077
16:23:55	77	Incierto	51.6 %	Sin etiqueta	Sin evaluar	7.190
16:23:57	78	Incierto	54.3 %	Sin etiqueta	Sin evaluar	6.088

TinyWear · ELECTRO 2026 · Ernesto Sifuentes de la Hoya

Cuestionario de evaluación

1. ¿Qué clases presentan mayor confusión y qué característica física podría explicarlo?
2. ¿Cómo cambia la interpretación de resultados cuando se prueba con otro participante?
3. ¿Qué diferencia existe entre exactitud de entrenamiento y desempeño en datos de prueba?
4. ¿Qué aporta la cuantización INT8 al despliegue en un microcontrolador?
5. ¿Qué ventaja tiene ejecutar la inferencia en el ESP32-S3 en lugar de enviar todos los datos a la nube?
6. ¿Qué modificaría primero para mejorar el sistema: dataset, ventanas, características o modelo? Justifique.

Conclusiones

Se completó el ciclo TinyML: datos etiquetados, segmentación temporal, entrenamiento, validación, optimización y despliegue. El resultado final es un sistema wearable capaz de convertir señales inerciales en una decisión de actividad. Más importante que obtener una cifra alta de exactitud es entender la calidad del dataset, los errores del modelo y las condiciones bajo las cuales el sistema generaliza.

Evidencia del participante

- Captura de la configuración del proyecto y las tres clases.
- Resultado de entrenamiento y matriz de confusión.
- Registro de desempeño con muestras de prueba.
- Evidencia del modelo desplegado o de la biblioteca generada.
- Captura de inferencia mostrando actividad y confianza.
- Respuestas al cuestionario y breve reflexión sobre generalización.

Anexo A. Productos generados durante P3

Producto	Uso
Proyecto Edge Impulse	Carga del dataset, configuración de ventanas, procesamiento, normalización, entrenamiento y validación del modelo.
Biblioteca Arduino: Taller_TinyWear_inferencing	Ejecución del modelo cuantizado INT8 y de la normalización exportada en el ESP32-S3.
Sketch: TinyWear_Inferencia_WIFI.ino	Adquisición de ventanas de 2 s, ejecución local del clasificador y envío de predicciones por Wi-Fi y UDP.
Archivos del sketch: TinyWearApp.cpp, TinyWearApp.h y TinyWearConfig.h	Implementación de la adquisición, inferencia y comunicación; configuración del kit, conexiones y parámetros de operación.
Interfaz: Interfaz_TinyWear_WIFI.py	Visualización de las tres probabilidades, actividad detectada, confianza y tiempos; registro de la validación en CSV.
Registro de pruebas: archivos .csv	Conservación de sujeto, actividad real y predicha, probabilidades, confianza, tiempos y resultado de la comparación. Se identifica el origen real o simulado de los datos.